

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



ỨNG DỤNG HỌC SÂU NHẬN DIỆN HÌNH ẢNH PHÁT
HIỆN ĐƯỜNG RAY XE LỬA HỒNG

Giảng viên hướng dẫn: TS. Trịnh Tấn Đạt

Tên	Mã số sinh viên
Lý Phúc Linh	3122411111
Nguyễn Quốc Hùng	31224111060
Trương Văn Hậu	31224111050
Nguyễn Công Tuyển	31224111239

TP.HỒ CHÍ MINH, tháng 12 năm 2025

DANH SÁCH THÀNH VIÊN DỰ ÁN

Tên	Mã số sinh viên
Lý Phúc Linh	3122411111
Nguyễn Quốc Hùng	31224111060
Trương Văn Hậu	31224111050
Nguyễn Công Tuyển	31224111239

LỜI CẢM ƠN

Nhóm chúng tôi xin chân thành cảm ơn TS. Trịnh Tấn Đạt – giảng viên hướng dẫn – đã tận tình hướng dẫn, góp ý phương pháp, kiểm tra và chỉnh sửa nội dung trong suốt quá trình thực hiện đề tài. Nhờ sự hướng dẫn và hỗ trợ quý báu của thầy, nhóm chúng tôi đã hoàn thiện báo cáo một cách chính xác, khoa học và đầy đủ. Chúng tôi xin bày tỏ lòng biết ơn sâu sắc về sự tận tâm, kiên nhẫn và những chia sẻ chuyên môn quý giá mà thầy đã dành cho nhóm.

Nhóm 4 người đồng ký

MỤC LỤC

DANH SÁCH THÀNH VIÊN DỰ ÁN.....	2
LỜI CẢM ƠN.....	3
MỤC LỤC	4
DANH MỤC HÌNH ẢNH.....	7
LỜI MỞ ĐẦU.....	8
CHƯƠNG 1 : TỔNG QUAN ĐỀ TÀI	9
1.1 Giới thiệu đề tài	9
1.2 Phương pháp triển khai đề tài	10
CHƯƠNG 2 : KIẾN THỨC NỀN TẢNG.....	11
2.1 Đại số tuyến tính.....	11
2.2 Giải tích	12
2.3 Xác suất thống kê	12
2.4 Khái niệm và kỹ thuật cốt lõi trong Deep Learning	13
CHƯƠNG 3 : MÔ HÌNH CONVOLUTIONAL NEURAL NETWORK (CNN) VÀ SUPPORT VECTOR MACHINE (SVM).....	19
3.1 Convolutional neural network	19
3.1.1 Giới thiệu	19
3.1.2 Kiến trúc	19
3.1.3 Resnet (Residual Network).....	22
3.1.4 Degradation Problem.....	23
3.1.5 Residual Learning (Học dư)	23
3.1.6 Triển khai Residual Block	25
3.1.7 Resnet50	27
3.2 Support Vector Machine.....	29
3.2.1 Giới thiệu	29

3.2.2 Nguyên lý hoạt động.....	29
3.2.3 Soft-margin SVM(cho dữ liệu có nhiễu).....	30
3.2.4 Kernel SVM (xử lý dữ liệu phi tuyến)	31
3.2.5 Bài toán đối ngẫu và support vectors	32
CHƯƠNG 4 : TRIỂN KHAI ỨNG DỤNG.....	33
4.1 Giới thiệu tập dữ liệu	33
4.2 CNN – Resnet50	34
4.2.1 Xử lý dữ liệu.....	34
4.2.2 Huấn luyện phase 1.....	35
4.2.3 Huấn luyện phase 2.....	36
4.2.4 Kết quả chạy mô hình.....	36
4.3 SVM – Kernel Support Vector Machine	38
4.3.1 Trích xuất đặc trưng.....	38
4.3.2 Tối ưu tham số và huấn luyện mô hình	40
4.3.3 Kết quả chạy mô hình.....	41
CHƯƠNG 5 : ĐÁNH GIÁ MÔ HÌNH.....	42
5.1 Đánh giá mô hình ResNet50.....	42
5.1.1 Accuracy trên tập test	42
5.1.2 Biểu đồ Loss và Accuracy trong quá trình huấn luyện	42
5.1.3 Đánh giá quá trình fine-tuning qua từng phase	42
5.2 Đánh giá mô hình Kernel SVM.....	43
5.2.1 Đánh giá định lượng	43
5.2.2 Khả năng tổng quát hóa của mô hình	43
5.3 Đánh giá và kết luận chung cho 2 mô hình	44
CHƯƠNG 6 : KẾT LUẬN.....	45
TÀI LIỆU THAM KHẢO	46

DANH MỤC HÌNH ẢNH

Hình 1 Các bộ lọc tích chập.....	14
Hình 2 Minh họa phép tích chập với stride = 1	15
Hình 3 Minh họa phép tích chập với stride = 2	16
Hình 4 Minh họa Feature Map được tạo ra sau nhiều lớp tích chập	20
Hình 5 Kết quả của phép Pooling tạo thành Pooled Feature Map.....	21
Hình 6 Flattening – chuyển Feature Map 2D thành vector 1D	22
Hình 7 Sơ đồ nguyên lý của khối Residual với kết nối tắt	24
Hình 8 Cấu trúc chi tiết một khối Residual cơ bản	24
Hình 9 Cơ chế so khớp kích thước đầu ra bằng Projection Shortcut	26
Hình 10 Kiến trúc tổng thể của mạng ResNet50	27
Hình 11 Minh họa siêu phẳng phân tách và lề mềm trong SVM	30
Hình 12 Ánh xạ phi tuyến sang không gian chiều cao bằng Kernel Trick.....	31
Hình 13 Ảnh minh họa Defective.....	33
Hình 14 Ảnh minh họa Non Defective	34
Hình 15 Kết quả huấn luyện phase 1	37
Hình 16 Kết quả huấn luyện phase 2	37
Hình 17 Kết quả tốt nhất của tập Test sau fine-tuning	38
Hình 18 TestReport SVM.....	41

LỜI MỞ ĐẦU

Trong bối cảnh phát triển hạ tầng giao thông và nhu cầu đảm bảo an toàn vận tải ngày càng cao, việc phát hiện sớm các hư hỏng trên đường ray là một yêu cầu cấp thiết. Truyền thống kiểm tra bằng phương pháp thủ công tốn nhiều thời gian, chi phí và có nguy cơ bỏ sót. Ứng dụng các phương pháp học sâu để tự động phát hiện lỗi trên hình ảnh đường ray hứa hẹn giảm thiểu chi phí, nâng cao tần suất kiểm tra và cải thiện độ an toàn.

Đề tài “Ứng dụng học sâu nhận diện hình ảnh phát hiện đường ray xe lửa hỏng” nghiên cứu giải pháp dựa trên mô hình học sâu (đặc biệt ResNet50) và so sánh/ứng dụng thêm phương pháp truyền thống như SVM trên các đặc trưng trích xuất được. Mục tiêu là xây dựng quy trình thu thập, tiền xử lý, huấn luyện và đánh giá mô hình để phát hiện hai nhãn chính: Defective và Non-defective, từ đó đưa ra khuyến nghị hỗ trợ công tác bảo trì dự đoán.

Báo cáo này trình bày lý thuyết, thiết kế hệ thống, thực nghiệm và đánh giá mô hình trên bộ dữ liệu Railway Track Fault Detection (nguồn: Kaggle). Nội dung được trình bày gồm: tổng quan đề tài, kiến thức nền tảng, mô hình convolutional neural network (CNN) và support vector machine (SVM), ứng dụng mô hình, đánh giá và kết luận.

CHƯƠNG 1 : TỔNG QUAN ĐỀ TÀI

1.1 Giới thiệu đề tài

Trong thời kỳ hội nhập và phát triển kinh tế như hiện nay, đặc biệt đối với các nước đang phát triển như Việt Nam, các tuyến đường sắt đóng vai trò chủ chốt trong vận chuyển và phân phối hàng hóa trong nước cũng như ngoài nước. Điểm mạnh của vận tải đường sắt là khối lượng vận tải khổng lồ, tạo ra lợi thế cạnh tranh lớn trên thị trường.

Việt Nam có địa hình trải dài từ Bắc vào Nam, với các đô thị tập trung sát biển nên mật độ dân cư tại những khu vực này khá cao. Do vậy, tuyến đường sắt Bắc – Nam có thể xem như một hệ thống giao thông vận tải huyết mạch và đóng vai trò quan trọng trong phát triển kinh tế cả nước.

Theo số liệu thống kê năm 2018, mạng lưới đường sắt quốc gia Việt Nam có 7 tuyến chính với tổng chiều dài gần 3.160 km, trong đó có 2.646 km đường chính tuyến và 514 km đường ga/nhánh, bao gồm 3 loại khổ ray, trong đó chủ yếu là khổ đường 1.000 mm (chiếm 84%), còn lại là khổ đường 1.435 mm (6%) và khổ đường lồng (9%).

Chi phí bảo trì và vận hành luôn là một bài toán lớn, đặc biệt với các hệ thống cơ sở hạ tầng phức tạp. Phương pháp bảo trì truyền thống, chủ yếu dựa vào sức người, bộc lộ nhiều hạn chế nghiêm trọng.

Việc kiểm tra thủ công không chỉ đòi hỏi một nguồn lực nhân công lớn, có chuyên môn cao mà còn cực kỳ tốn thời gian. Các đội kỹ thuật phải trực tiếp khảo sát hiện trường, đôi khi trong những điều kiện nguy hiểm (như trên cao, đường hầm, hoặc gần thiết bị đang vận hành). Quy trình này không chỉ chậm chạp, tốn kém mà còn tiềm ẩn rủi ro sai sót do yếu tố chủ quan của con người, dễ bỏ qua các hư hỏng nhỏ hoặc dấu hiệu xuống cấp sớm.

Để khắc phục những nhược điểm này, việc ứng dụng công nghệ tự động hóa là một xu hướng tất yếu. Trong đó, Deep Learning (Học sâu), đặc biệt là công nghệ nhận diện hình ảnh, đang mở ra một cuộc cách mạng trong lĩnh vực bảo trì.

Thay vì con người phải "nhìn" trực tiếp, các hệ thống camera, drone (máy bay không người lái) hoặc robot có thể được trang bị để tự động thu thập dữ liệu hình ảnh hoặc video liên tục. Các mô hình Deep Learning, sau khi được huấn luyện với hàng triệu hình ảnh, có khả năng "nhìn" và phân tích dữ liệu này với độ chính xác vượt trội.

- Các vết nứt trên đường ray.
- Dấu hiệu gỉ sét, ăn mòn trên các cây cầu, đường ống.
- Sự mài mòn hoặc sai lệch của các linh kiện máy móc.

Việc áp dụng Deep Learning giúp chuyển đổi toàn bộ quy trình: thay vì phản ứng hỏng đầu sửa đó, chúng ta có thể bảo trì dự đoán hỏng hóc trước khi nó xảy ra. Điều này không chỉ giúp giảm thiểu thời gian ngừng hoạt động, tiết kiệm đáng kể chi phí nhân công mà còn tăng cường an toàn và kéo dài tuổi thọ của công trình.

1.2 Phương pháp triển khai đề tài

Đề tài này sử dụng tập dữ liệu Railway Track Fault Detection trên Kaggle (dung lượng 2.14 GB) gồm các hình ảnh về đường ray xe lửa. Mục tiêu là huấn luyện mô hình Deep Learning học các đặc trưng về lỗi, từ đó đưa ra dự đoán về tình trạng đường ray, nhằm hỗ trợ con người ra quyết định sửa chữa hay tiếp tục sử dụng.

Trong đề tài này, các mô hình chính được sử dụng là Convolutional Neural Network (CNN) và Support Vector Machine (SVM). Cụ thể, tập trung vào hai biến thể:

- Mô hình ResNet50: Một biến thể mạnh mẽ của CNN, được sử dụng để nhận diện hình ảnh.
- Mô hình Kernel SVM: Một biến thể của Support Vector Machine, được sử dụng để phân loại ảnh đường ray.

Mục tiêu cuối cùng của đề tài là đánh giá tính ứng dụng thực tế và tính khả thi khi triển khai mô hình này ở quy mô lớn.

CHƯƠNG 2 : KIẾN THỨC NỀN TẢNG

Toán học là kiến thức cốt lõi của các thuật toán học sâu. Nếu không có đại số tuyến tính, chúng ta sẽ không thể biểu diễn được dữ liệu, trọng số, các phép biến đổi trong mạng nơ-ron bằng các vector, các ma trận, tensor,... Nếu không có giải tích, chúng ta sẽ không thể giúp mô hình “học” thông qua đạo hàm, gradient và lan truyền ngược (backpropagation) để tối ưu hàm mất mát, tìm ra trọng số mô hình. Nếu không có xác suất thống kê, chúng ta sẽ không thể xử lý dữ liệu đầu vào, không thể chuẩn hóa dữ liệu, xử lý nhiễu và phân phối trong ảnh.

2.1 Đại số tuyến tính

Đại số tuyến tính là cầu nối quan trọng giúp chúng ta biểu diễn dữ liệu dưới dạng toán học và thực hiện các phép toán biến đổi

Vector : một mảng các giá trị vô hướng.

$$x = [x_1, x_2, x_3, \dots, x_n]$$

Ma trận : một mảng chữ nhật các giá trị vô hướng có m hàng và n cột, gọi là ma trận $m \times n$.

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}$$

Tensor : cách gọi của 1 ma trận có nhiều hơn 2 chiều.

Ví dụ : tensor 3 chiều $m \times n \times k$, nghĩa là xếp k lần ma trận kích thước $m \times n$

Kernel : 1 ma trận vuông kích thước $k \times k$, k là số lẻ.

$$W = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Tổ hợp tuyến tính và phép biến đổi : Phép toán cơ bản của một nơ-ron là một phép biến đổi tuyến tính, là một tổ hợp tuyến tính của các đầu vào

$$y = w_1x_1 + w_2x_2 + \dots + b$$

Kết quả sau đó được đưa qua **hàm kích hoạt phi tuyến** (Activation Function). giúp mô hình biểu diễn phức tạp hơn.

Chuẩn (Norms): Chuẩn được dùng để đo “độ lớn” của một vector $\ell_1 \text{ norm} = \|w\|_1 = \sum_i |w_i|$, đo tổng giá trị tuyệt đối của các phần tử

$\ell_2 \text{ norm} = \|w\|_2 = \sqrt{\sum_i w_i^2}$, đo khoảng cách Euclid từ gốc đến w

2.2 Giải tích

Nếu giải tích là xây dựng mô hình, thì giải tích là cách mà mô hình học các trọng số quan trọng

Đạo hàm : $f'(x)$ đo lường tốc độ thay đổi của một hàm số $f(x)$

Đạo hàm riêng : Với hàm có nhiều biến $f(x_1, x_2, x_3 \dots x_n)$, $\frac{\partial f}{\partial x_i}$ đạo hàm theo biến thứ i trong hàm nhiều biến.

Gradient : ∇f là vector chứa tất cả các đạo hàm riêng, chỉ hướng dốc nhất (tăng nhanh nhất), thường được dùng để cập nhật trọng số . Với hàm có nhiều biến $f(x_1, x_2, x_3 \dots x_n)$

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial x_3} \dots \frac{\partial f}{\partial x_n} \right]$$

Quy tắc chuỗi (Chain Rule) : Là cơ sở của Backpropagation

$$\text{Nếu } z = g(y) \text{ và } y = f(x) \text{ thì } \frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

Hàm mất mát (Loss Function) : $\mathcal{L}$ hoặc $J(\theta)$ Hàm số đo lường độ chính xác của mô hình, tức là sai số giữa dự đoán $\hat{y}$ và giá trị thực y . Càng thấp thì dự đoán càng chính xác. Cốt lõi của thuật toán học máy là giải bài toán tối ưu hàm mất mát.

$$\text{(MSE)} \mathcal{L} = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$$

$$\text{(Cross - Entropy)} \mathcal{L} = - \sum_i y_i \log(\hat{y}_i)$$

$$\text{(Binary Cross)} \mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

2.3 Xác suất thống kê

Xác suất thống kê sẽ giúp chúng ta khởi tạo mô hình, tối ưu hóa và hiểu dữ liệu.

Phân phối chuẩn (Gaussian Distribution) : phân phối chuẩn mô tả nhiều và giá trị đặc trưng

$$\mathcal{N}(0, \sigma^2)$$

$$\text{Kỳ vọng} : \mathbb{E}[X] = \frac{1}{n} \sum_i x_i$$

$$\text{Phương sai} : \text{Var}(X) = \frac{1}{n} \sum_i (x_i - \mathbb{E}[X])^2$$

2.4 Khái niệm và kỹ thuật cốt lõi trong Deep Learning

Feature vector : là các vector đặc trưng được trích xuất từ 1 đối tượng (ảnh, văn bản ,..)

Overfitting và Underfitting : Overfitting là khi 1 mô hình khớp với tập huấn luyện mà không khớp với 2 tập còn lại. Underfitting là khi mô hình quá đơn giản, không học được quy luật.

Regularization : là kĩ thuật giảm overfitting bằng cách giảm độ phức tạp mô hình.

Phép cộng phần tử (Element-wise addition) : Đây là phép toán cốt lõi trong ResNet, cho phép tín hiệu và gradient đi qua mạng mà không suy giảm.

$$Y = f(X) + X$$

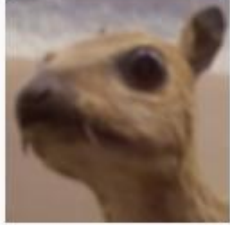
với $f(X)$ là đầu ra của chuỗi các phép tích chập và chuẩn hóa.

Phép tích chập (Convolution) : Phép toán cốt lõi của CNN. Một kernel nhỏ "trượt" qua đầu vào, thực hiện phép nhân theo từng phần tử và cộng lại để tạo ra một điểm ảnh trên bản đồ đặc trưng. Kí hiệu $\otimes$

$$\begin{array}{ccccc}
 \begin{array}{ccc} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{array} & \begin{array}{cc} 0 & 0 \\ 1 & 0 \\ 1 & 1 \end{array} & \otimes & \begin{array}{ccc} 0 & 1 & 0 \\ 1 & 0 & 1 \end{array} & = & \begin{array}{ccc} 4 & \dots & \dots \\ \dots & \dots & \dots \\ \dots & \dots & \dots \end{array} \\
 X & W & & Y
 \end{array}$$

$$\begin{aligned}
 y_{11} &= \text{sum}(A \otimes W) = X_{11} * w_{11} + X_{12} * w_{12} + X_{13} * w_{13} + X_{21} * w_{21} + X_{22} * \\
 &w_{22} + X_{23} * w_{23} + X_{31} * w_{31} + X_{32} * w_{32} + X_{33} * w_{33} = 4
 \end{aligned}$$

Ý nghĩa của phép này là để trích xuất đặc trưng của ảnh bằng cách trượt qua các kernel khác nhau

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	

Hình 1 Các bộ lọc tích chập

Padding : là một kĩ thuật thêm viền giá trị 0 vào ma trận output của phép Convolution Padding=k nghĩa là thêm k vector 0 vào mỗi phía (trên, dưới, trái, phải) của ma trận.

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

Hình 2 Minh họa phép tích chập với $stride = 1$

Stride : bước nhảy, $stride=k$ ($k > 1$) thì ta chỉ thực hiện phép tính convolution trên các phần tử $x_{1+i*k, 1+j*k}$, thường dùng để giảm kích thước của ma trận sau phép tính convolution

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

Hình 3 Minh họa phép tích chập với stride = 2

Chuẩn hóa dữ liệu (Data Normalization) : là bước đưa dữ liệu đầu vào về cùng 1 thang giá trị μ là trung bình, σ là độ lệch chuẩn

$$x' = \frac{x - \mu}{\sigma}$$

Early Stopping: dừng huấn luyện khi hàm mất mát của tập Validation không giảm thêm sau k epoch

Dropout : Trong quá trình huấn luyện, ngẫu nhiên vô hiệu hóa một số nơ-ron để giảm sự phụ thuộc giữa các nơ-ron và tăng tính tổng quát

$$h' = h_i \cdot r_i, r_i \sim \text{Bernoulli}(p)$$

L2 Regularization (Weight Decay) : “phạt” trọng số, ép mô hình học đặc trưng đơn giản

$$L' = L + \lambda \|W\|_2^2$$

Vanishing gradient và Exploding gradient : Khi mạng rất sâu, việc nhân liên tục các đạo hàm (theo quy tắc chuỗi) có thể khiến gradient tiến về 0 (biến mất) hoặc trở nên cực lớn (bùng nổ), làm cho mô hình không thể học.

Weight Initialization : Khởi tạo trọng số , giúp mô hình ở trạng thái cân bằng, tránh Vanishing gradient và Exploding gradient.

$$Var(W) = \frac{2}{n_{in}}$$

Chuẩn hóa batch (Batch Normalization) : giúp ổn định gradient và tăng tốc huấn luyện μ_B, σ_B^2 là trung bình và phương sai theo batch , γ, β là các tham số học được

$$y = \gamma \hat{x} + \beta$$

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Gradient Descent : 1 phương pháp tối ưu hàm mất mát, tìm θ^* sao cho hàm mất mát $L(\theta)$ nhỏ nhất η (learning rate) là tốc độ học, $\nabla_{\theta} \mathcal{L}$ là gradient

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$

Lan truyền ngược (Backpropagation) : thuật toán tính đạo hàm của hàm mất mát theo từng trọng số, thứ tự từ output tới input. Giải pháp cho vấn đề Vanishing Gradient.

$$\frac{\partial L}{\partial X} = \frac{\partial fL}{\partial Y} \left(1 + \frac{\partial F}{\partial X}\right)$$

Data Augmentation : kĩ thuật tăng cường dữ liệu, nhân rộng dữ liệu huấn luyện mà không thay đổi nhãn (flip, rotate, crop,...)

Epoch : 1 lần mô hình học qua toàn bộ dữ liệu

Learning rate schedule : giảm tốc độ học sau 1 thời gian giúp ổn định mô hình

Transfer learning : Tận dụng kiến thức từ mô hình đã học trên tập dữ liệu lớn để huấn luyện trên tập nhỏ

$$f = F_{pretrained}(X)$$

$$\hat{y} = g_{new}(f)$$

$F_{pretrained}$ phần convolution (trích đặc trưng).

g_{new} phần classifier mới (được huấn luyện lại).

Fine tuning : 1 kỹ thuật transfer learning, đóng băng các lớp đầu (học đặc trưng cơ bản như cạnh, góc), huấn luyện lại các lớp cuối (đặc trưng đặc thù cho đường ray).

Neural Network (NN) : là một mô hình tính toán lấy cảm hứng từ cấu trúc và hoạt động của bộ não sinh học. Đơn vị cơ bản của não bộ là các nơ-ron thần kinh, và trong NN, đơn vị cơ bản tương ứng được gọi là Perceptron hoặc nơ-ron nhân tạo.

Input layer → *Hidden layer* → *Output layer*

Lớp Đầu vào (Input Layer): Nhận dữ liệu thô (ví dụ: các pixel của một bức ảnh được duỗi thẳng thành một vector dài).

Lớp Ẩn (Hidden Layer): Nằm giữa lớp đầu vào và đầu ra. Đây là nơi diễn ra hầu hết các phép tính toán và trích xuất đặc trưng. Một mạng có thể có một hoặc nhiều lớp ẩn; các mạng có nhiều lớp ẩn được gọi là "học sâu" (Deep Learning).

Lớp Đầu ra (Output Layer): Sản xuất kết quả cuối cùng của mô hình (ví dụ: xác suất ảnh này là "Lỗi" hay "Bình thường").

Precision : Tỷ lệ dự đoán đúng trong số các dự đoán **đương tính** mà mô hình đã đưa ra. Precision cao nghĩa là mô hình rất ít báo nhầm “có lỗi” khi thực tế không có.

Recall : Tỷ lệ dự đoán đúng trong số các mẫu **đương tính thực sự**. Recall cao nghĩa là mô hình ít bỏ sót lỗi.

F1-score : Trung bình điều hòa giữa Precision và Recall. Dùng để đánh giá tổng hợp khi cần cân bằng giữa "báo nhầm" và "bỏ sót".

Support : Số lượng mẫu thật sự thuộc về một lớp nào đó trong tập dữ liệu (ví dụ có bao nhiêu ảnh là “Lỗi”, bao nhiêu ảnh “Bình thường”).

Train : Giá trị đo trên tập huấn luyện — nơi mô hình học các đặc trưng.

Val : Giá trị đo trên tập validation — dùng để kiểm tra mô hình có bị overfit không.

LR (Learning Rate) : Tốc độ học, quyết định mức độ điều chỉnh trọng số sau mỗi bước tối ưu. LR quá lớn → dễ làm mô hình dao động; quá nhỏ → học rất chậm.

Công thức 1 nơ ron trong Hidden layer : x_i là đầu vào , w_i trọng số tương ứng x_i , b là hệ số bias, $f(x)$ là hàm kích hoạt phi tuyến , a là đầu ra nơ ron

$$z = \sum_{i=1}^n w_i x_i + b$$
$$a = f(x)$$

CHƯƠNG 3 : MÔ HÌNH CONVOLUTIONAL NEURAL NETWORK (CNN) VÀ SUPPORT VECTOR MACHINE (SVM)

3.1 Convolutional neural network

3.1.1 Giới thiệu

Mạng nơ-ron tích chập (Convolutional Neural Network – CNN) là một loại mô hình học sâu được thiết kế để xử lý dữ liệu có cấu trúc không gian như ảnh, video hoặc chuỗi thời gian. Khác với mạng nơ-ron truyền thống (Fully Connected Neural Network), CNN khai thác mối liên hệ cục bộ giữa các điểm dữ liệu thông qua phép tích chập (convolution), giúp giảm số lượng tham số và tăng khả năng tổng quát hóa.

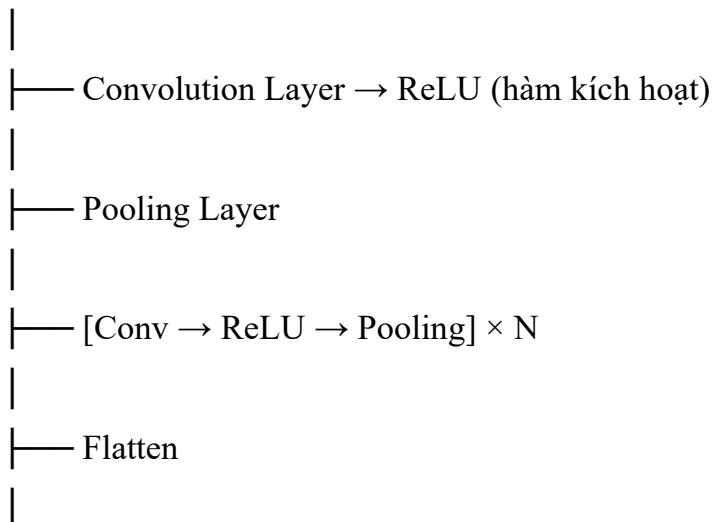
CNN là nền tảng của hầu hết các ứng dụng trong thị giác máy tính (computer vision) như:

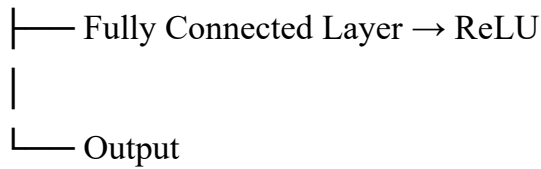
- Nhận dạng ảnh (Image Classification)
- Phát hiện vật thể (Object Detection)
- Phân đoạn ảnh (Image Segmentation)
- Phát hiện lỗi bề mặt, hư hại công trình, hoặc đường ray (như trong đề tài này)

3.1.2 Kiến trúc

Kiến trúc mô hình CNN điển hình được cấu thành từ việc xếp chồng nhiều lớp xử lý khác nhau.

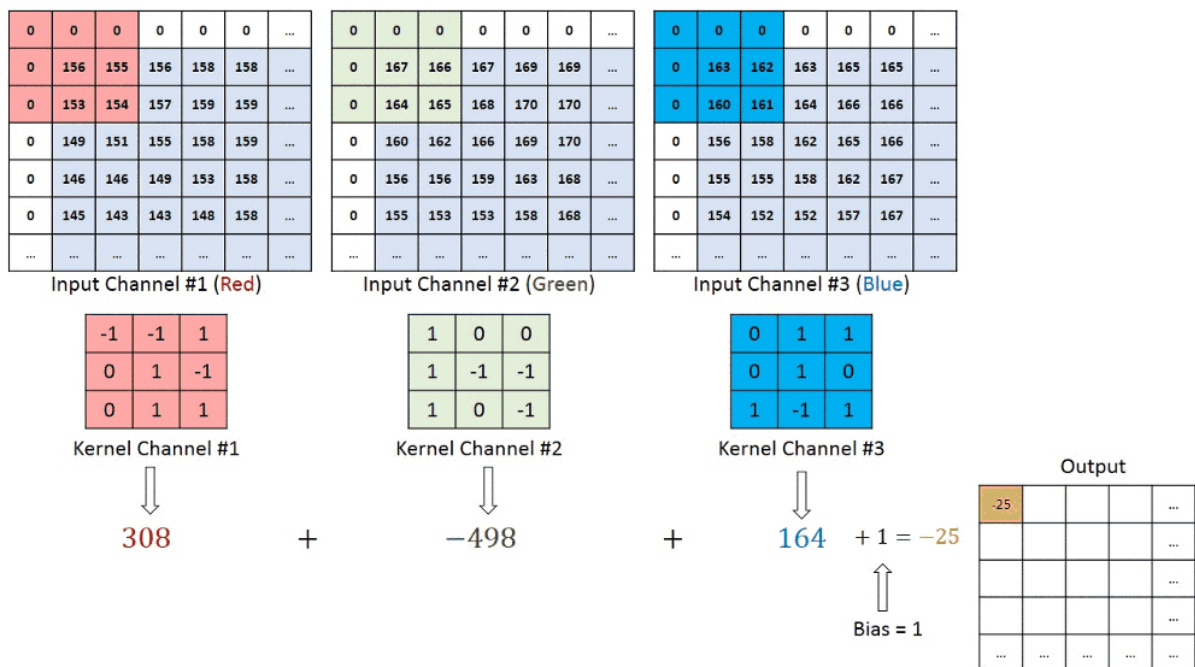
Input





Convolution Layer (Lớp tích chập): Đây là khối xây dựng cơ bản và quan trọng nhất của CNN. Thay vì kết nối mọi nơ-ron đầu vào với mọi nơ-ron đầu ra, lớp tích chập sử dụng các kernel nhỏ để quét qua toàn bộ ảnh. Kernel sẽ trượt trên toàn bộ ảnh đầu vào. Tại mỗi vị trí, nó thực hiện phép nhân theo từng phần tử (element-wise multiplication) với vùng ảnh mà nó bao phủ rồi tính tổng các giá trị đó. Kết quả của phép tính tổng này là một giá trị duy nhất trong ma trận đầu ra.

Feature Map (Bản đồ Đặc trưng): Toàn bộ ma trận đầu ra được tạo ra bởi một bộ lọc duy nhất được gọi là "bản đồ đặc trưng". Nó biểu thị vị trí và cường độ của đặc trưng mà bộ lọc đó tìm thấy trong ảnh. Một lớp tích chập thường có nhiều bộ lọc (ví dụ: 32, 64) để học nhiều loại đặc trưng khác nhau cùng một lúc.



Hình 4 Minh họa Feature Map được tạo ra sau nhiều lớp tích chập

Bản thân phép tích chập là một phép toán tuyến tính. Nếu chỉ xếp chồng các lớp tích chập, toàn bộ mạng vẫn sẽ là một mô hình tuyến tính, không đủ khả năng học các mối quan hệ phức tạp trong thế giới thực. Do đó, sau mỗi lớp tích chập, một hàm kích hoạt phi tuyến được áp dụng cho mọi phần tử trong bản đồ đặc trưng.

ReLU (Rectified Linear Unit): Là hàm kích hoạt phổ biến nhất trong các CNN hiện đại. Công thức của nó cực kỳ đơn giản: $f(x) = \max(0, x)$

ReLU giúp giải quyết vấn đề vanishing gradient cho phép mạng học nhanh hơn và huấn luyện được các kiến trúc rất sâu.

Pooling Layer (Lớp gộp) : Sau khi các đặc trưng được phát hiện bởi lớp tích chập và hàm kích hoạt, Lớp Gộp có nhiệm vụ giảm chiều dữ liệu (down-sampling) một cách có hệ thống, làm cho biểu diễn dữ liệu trở nên nhỏ gọn và dễ quản lý hơn.

- **Giảm tải tính toán:** Giảm kích thước không gian (chiều cao, chiều rộng) của bản đồ đặc trưng, từ đó giảm số lượng tham số và phép tính ở các lớp sau.
- **Tạo tính Bất biến (Invariance):** Giúp mô hình có khả năng bất biến với các dịch chuyển nhỏ. Ví dụ, dù đặc trưng (một con mắt) nằm ở pixel (10, 10) hay (11, 11), lớp gộp vẫn sẽ cho ra kết quả tương tự. Điều này giúp mô hình tập trung vào sự hiện diện của đặc trưng chứ không phải vị trí chính xác của nó.

Các loại Gộp phổ biến:

- **Max Pooling:** Phương pháp phổ biến nhất. Nó lấy một cửa sổ (và chỉ giữ lại giá trị **lớn nhất** trong cửa sổ đó. Điều này có nghĩa là nó giữ lại tín hiệu mạnh nhất, nổi bật nhất của đặc trưng.
- **Average Pooling:** Lấy giá trị **trung bình** của tất cả các giá trị trong cửa sổ. Phương pháp này làm mượt dữ liệu, nhưng có thể làm loãng các tín hiệu đặc trưng mạnh.

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

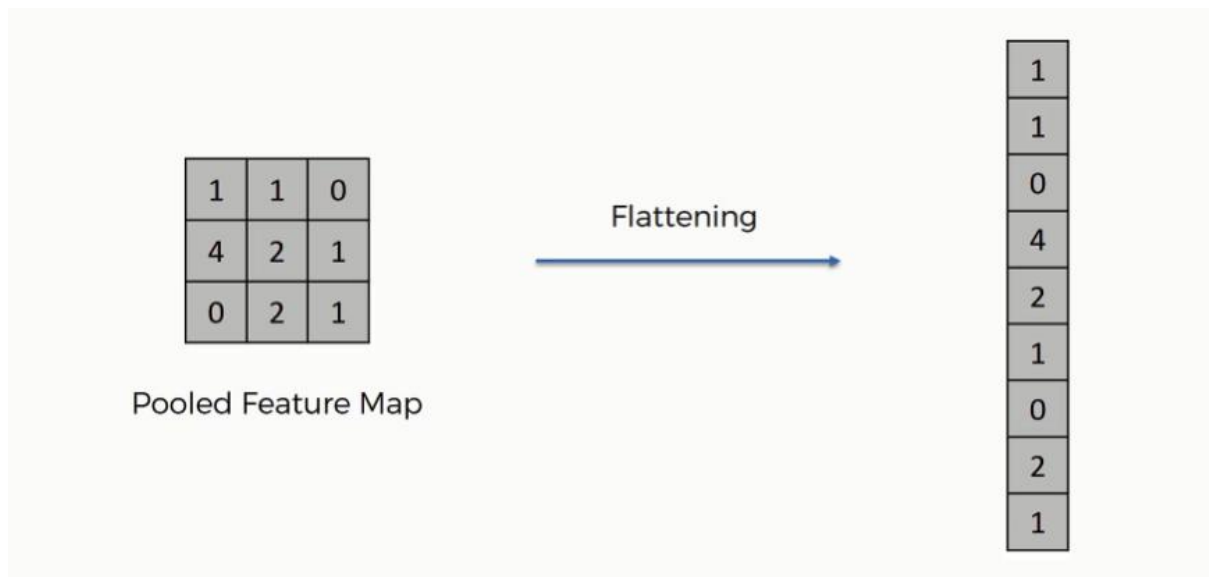
3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

Hình 5 Kết quả của phép Pooling tạo thành Pooled Feature Map

Fully Connected Layer (Lớp Kết nối Đầy đủ) : Sau khi đi qua nhiều chuỗi Convolution Layer và Pooling Layer, mô hình đã học được một tập hợp các bản đồ đặc

trung ở mức độ cao (high-level). Các bản đồ này vẫn ở dạng 3D (chiều cao x chiều rộng x số kênh). Để đưa ra quyết định phân loại cuối cùng, chúng ta cần chuyển đổi dữ liệu 3D này thành một vector 1D:

- **Flatten Layer (Lớp Làm phẳng):** Thực hiện một thao tác đơn giản là "đuỗi thẳng" khối đặc trưng 3D thành một vector 1D duy nhất.



Hình 6 Flattening – chuyển Feature Map 2D thành vector 1D

- **Fully Connected Layer (Lớp Kết nối Đầy đủ):** Vector 1D này sau đó được đưa vào một hoặc nhiều Lớp Kết nối Đầy đủ, hoạt động giống hệt như Mạng Nơ-ron Nhân tạo (ANN). Lớp này có nhiệm vụ tổng hợp tất cả các thông tin đặc trưng đã được trích xuất từ ảnh để đưa ra quyết định.
- **Output Layer (Lớp Đầu ra):** Lớp FC cuối cùng sẽ có số nơ-ron bằng với số lớp (class) mà chúng ta muốn phân loại (ví dụ: 2 nơ-ron cho bài toán "Lỗi" và "Bình thường"). Một hàm kích hoạt như Softmax (cho phân loại nhiều lớp) hoặc Sigmoid (cho phân loại nhị phân) thường được sử dụng ở lớp cuối cùng để chuyển đổi điểm số của nơ-ron thành xác suất.

Tóm lại, một mô hình CNN hoạt động bằng cách sử dụng các Convolution Layer và Pooling Layer để tự động học và trích xuất các đặc trưng phân cấp từ ảnh sau đó sử dụng các Fully Connected Layer để phân loại dựa trên các đặc trưng đó.

3.1.3 Resnet (Residual Network)

Sau thành công của các kiến trúc CNN như AlexNet và VGG, giới nghiên cứu nhận thấy một xu hướng rõ ràng: các mô hình "sâu hơn" (có nhiều lớp hơn) thường cho

kết quả tốt hơn. Tuy nhiên, việc đơn giản là xếp chồng thêm nhiều lớp (ví dụ: từ 20 lên 50, 100 lớp) lại dẫn đến một vấn đề nan giải.

3.1.4 Degradation Problem

Một cách trực quan, người ta kỳ vọng một mô hình 50 lớp sẽ có hiệu năng ít nhất là bằng một mô hình 20 lớp. Trong trường hợp xấu nhất, 30 lớp thêm vào có thể học một **identity mapping** (ánh xạ đồng nhất) – tức là chỉ cần trả ra đúng đầu vào của nó mà không làm gì cả, khi đó hiệu năng sẽ tương đương.

Tuy nhiên, thực tế lại không như vậy. Các mô hình rất sâu thường bị **degradation** (suy giảm): khi số lớp tăng lên, cả training error và testing error đều tăng lên. Điều này cho thấy mô hình không chỉ bị overfitting, mà còn không thể tối ưu được chính nó.

- Vanishing Gradient : Càng nhiều lớp, gradient bị giảm khi lan truyền ngược
- Khó khăn trong việc học identity mapping: Các hàm kích hoạt phi tuyến và các lớp tích chập khiến việc học một ánh xạ $H(x) = x$ (chỉ trả lại đầu vào) trở nên cực kỳ khó khăn.

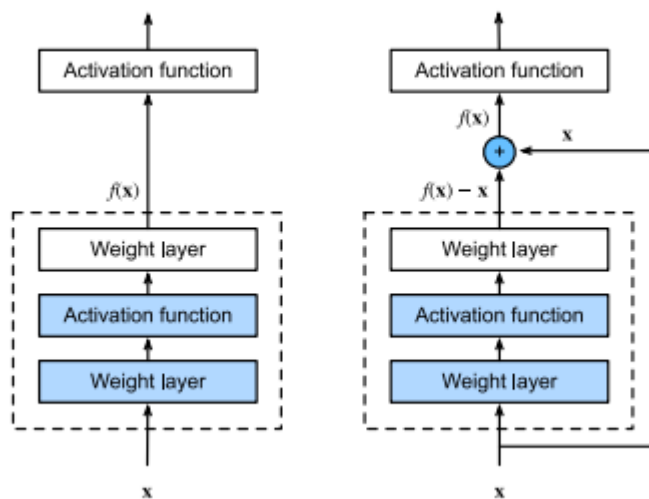
Giải pháp cho vấn đề đó, người ta tìm ra Residual Learning (Học dư) và Residual Block (Khối dư)

3.1.5 Residual Learning (Học dư)

ResNet, được giới thiệu bởi Kaiming He và cộng sự vào năm 2015, đã giải quyết triệt để vấn đề này bằng một ý tưởng: **Skip Connection (Kết nối tắt)**. Thay vì bắt một khối các lớp học một ánh xạ $H(x)$ mong muốn, ResNet bắt chúng học một **residual mapping (ánh xạ dư)**, ký hiệu là $F(x)$.

Kiến trúc của một **Residual Block** được định nghĩa như sau:

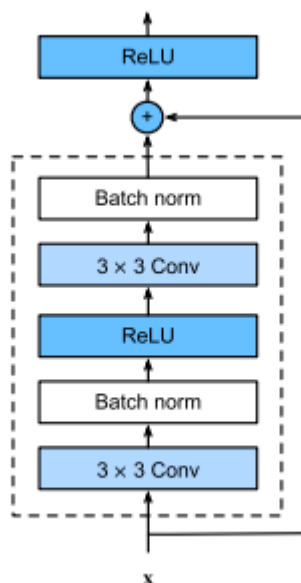
- Đầu vào của khối là x
- Đầu ra mong muốn của khối là $H(x)$.
- Thay vì học $H(x)$ trực tiếp, ta định nghĩa $F(x) = H(x) - x$. Đây chính là phần residual (dư) – tức là phần chênh lệch mà các lớp này cần học thêm vào x để ra được $H(x)$.
- Từ đó suy ra, ánh xạ cuối cùng của khối là: $H(x) = F(x) + x$



Hình 7 Sơ đồ nguyên lý của khối Residual với kết nối tắt

Cơ chế hoạt động:

- Đầu vào x được đưa vào một nhánh tính toán $F(x)$ (thường gồm 2-3 lớp tích chập, Batch Norm và ReLU).
- Cùng lúc đó đầu vào x được đưa qua một skip connection (một đường nối thẳng, không qua phép biến đổi nào, hoặc chỉ biến đổi 1×1 để khớp kích thước) đến cuối khối.
- Đầu ra của nhánh $F(x)$ và đầu ra của nhánh tắt x được cộng lại với nhau theo từng phần tử (element-wise addition).



Hình 8 Cấu trúc chi tiết một khối Residual cơ bản

- **Giải quyết vấn đề identity mapping:** Nếu trong trường hợp tối ưu, các lớp này không cần làm gì cả (tức $H(x) = x$), mô hình chỉ cần học $F(x) = 0$.
- **Giải quyết vấn đề Vanishing Gradient:** Trong quá trình lan truyền ngược, phép cộng $H(x) = F(x) + x$ cho phép gradient đi qua hai con đường. Một đường đi qua nhánh $F(x)$ (như bình thường) và một đường đi thẳng qua nhánh x . Con đường này cho phép gradient truyền thẳng về các lớp đầu tiên mà không bị suy giảm, giúp mạng có thể huấn luyện ngay cả khi rất sâu.

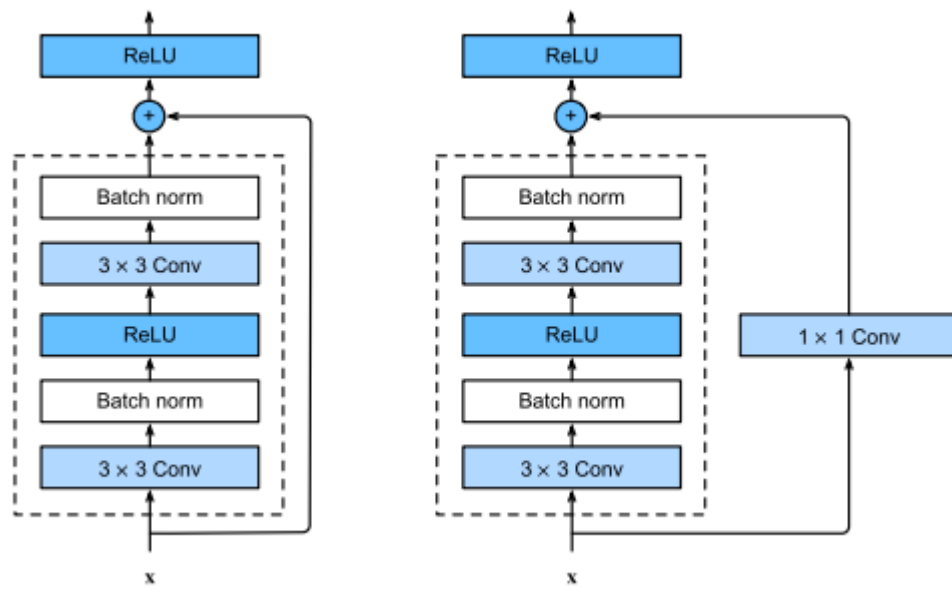
Nhờ kiến trúc này, các nhà nghiên cứu đã có thể huấn luyện thành công các mạng có độ sâu 152 lớp, 1000 lớp và hơn thế nữa, đạt được độ chính xác vượt trội trên nhiều bộ dữ liệu.

3.1.6 Triển khai Residual Block

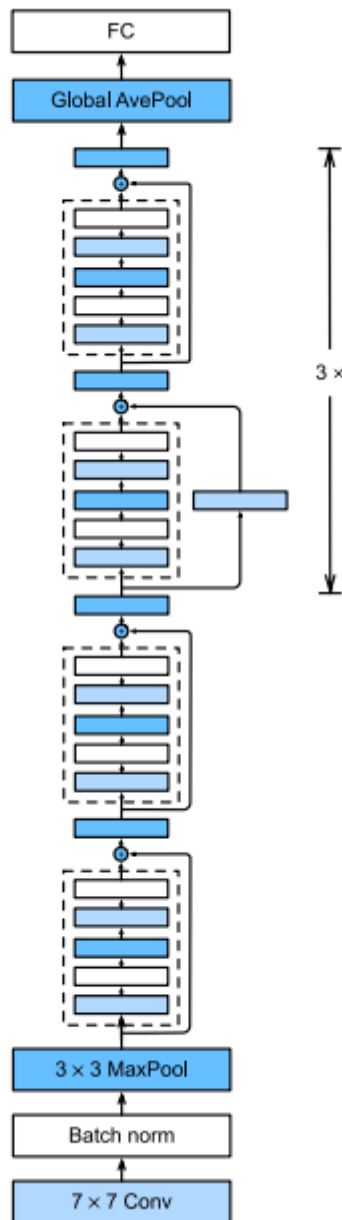
Để triển khai khái niệm học dư, ResNet kế thừa triết lý thiết kế sử dụng các Convolution Layer 3x3 đồng nhất. Một Residual Block cơ bản thường được cấu tạo như sau:

- Nhánh chính $F(x)$ bao gồm hai Convolution Layer 3x3 được xếp chồng lên nhau, giữ nguyên số kênh đầu ra.
- Mỗi lớp tích chập này được theo sau bởi một lớp Batch Normalization và một hàm ReLU.
- Sau khi dữ liệu đi qua hai Convolution Layer này, Skip connection sẽ cộng đầu vào x ban đầu vào kết quả $F(x)$.
- Tổng $H(x) = F(x) + x$ sau đó sẽ được đưa qua hàm ReLU cuối cùng của khối.

Thiết kế này yêu cầu đầu ra của nhánh Convolution $F(x)$ phải có cùng hình dạng (chiều cao, chiều rộng, và số kênh) với đầu vào x để phép cộng có thể thực hiện được. Trong trường hợp cần thay đổi (tăng số kênh hoặc giảm kích thước), kết nối tắt x sẽ phải đi qua một Convolution Layer 1x1 bổ sung (projection shortcut). Lớp này có nhiệm vụ chuyển đổi x thành hình dạng mong muốn trước khi thực hiện phép cộng, đảm bảo tính tương thích về kích thước.



Hình 9 Cơ chế so khớp kích thước đầu ra bằng Projection Shortcut



Hình 10 Kiến trúc tổng thể của mạng ResNet50

3.1.7 Resnet50

ResNet50 là một trong những kiến trúc mạng nơ-ron tích chập (CNN) phổ biến và có ảnh hưởng nhất, thuộc họ ResNet . Con số "50" thể hiện rằng mô hình này có tổng cộng 50 lớp có thể huấn luyện (bao gồm 49 lớp tích chập và 1 lớp kết nối đầy đủ cuối cùng). Nó được coi là một cột mốc quan trọng vì đã cân bằng thành công giữa deep (độ sâu) của mạng và hiệu quả tính toán, trở thành một lựa chọn mặc định cho rất nhiều bài toán thị giác máy tính.

Khi các mạng ResNet trở nên rất sâu (như 50, 101, hay 152 lớp), Residual block cơ bản bắt đầu trở nên tốn kém về mặt tính toán.

Để giải quyết vấn đề này, ResNet50 (và các mạng sâu hơn) sử dụng một thiết kế Residual block được tối ưu hóa, gọi là Bottleneck Block (Khối "Nút cổ chai"). Khối này hiệu quả hơn về mặt tham số và tính toán.

Thay vì dùng 2 lớp 3×3 , Bottleneck Block sử dụng một chuỗi 3 Convolution Layer

- Convolution Layer **1x1 (Giảm chiều)**: Lớp đầu tiên này nhận đầu vào có số kênh lớn (ví dụ: 256 kênh) và áp dụng các bộ lọc 1×1 để *giảm* số lượng kênh xuống một mức nhỏ hơn (ví dụ: 64 kênh). Đây chính là phần "nút cổ chai", giúp giảm tải tính toán cho lớp 3×3 tiếp theo.
- Convolution Layer **3x3 (Xử lý không gian)**: Convolution Layer 3×3 tiêu chuẩn thực hiện việc học đặc trưng không gian. Nhưng giờ đây, nó chỉ hoạt động trên đầu vào có 64 kênh (đã được giảm) thay vì 256 kênh, khiến phép toán này nhanh hơn đáng kể.
- Convolution Layer **1x1 (Phục hồi chiều)**: Lớp cuối cùng này nhận đầu ra 64 kênh từ lớp 3×3 và sử dụng các bộ lọc 1×1 để *mở rộng* (phục hồi) số kênh trở lại 256. Việc này đảm bảo rằng đầu ra $F(x)$ có cùng số kênh với đầu vào x ban đầu, để có thể thực hiện phép cộng $F(x) + x$ của skip connection.

Một Khối Residual Block cơ bản với 256 kênh sẽ có hai lớp $3 \times 3 \times 256$. Trong khi đó, Bottleneck Block thay thế điều này bằng các lớp $1 \times 1 \times 64$, $3 \times 3 \times 64$, $1 \times 1 \times 256$. Thiết kế này cho phép ResNet50 có độ sâu lớn hơn nhiều mà vẫn giữ chi phí tính toán trong tầm kiểm soát.

Kiến trúc ResNet50 đầy đủ được tổ chức thành 5 giai đoạn (phases):

- **Phase 1 (Conv1)**: Một lớp tích chập 7×7 lớn ban đầu và một lớp Max Pooling 3×3 để nhanh chóng giảm kích thước không gian và nắm bắt các đặc trưng tổng quát.
- **Phase 2 (Conv2)**: Một chuỗi gồm 3 Bottleneck Block.
- **Phase 3 (Conv3)**: Một chuỗi gồm 4 Bottleneck Block.
- **Phase 4 (Conv4)**: Một chuỗi gồm 6 Bottleneck Block.
- **Phase 5 (Conv5)**: Một chuỗi gồm 3 Bottleneck Block

Sau 5 giai đoạn này, một lớp **Global Average Pooling (Gộp Trung bình Toàn cục)** được sử dụng để lấy trung bình của từng feature map, tạo ra một vector đặc trưng duy nhất. Vector này sau đó được đưa vào một Fully Connection Layer cuối cùng để đưa ra dự đoán.

3.2 Support Vector Machine

3.2.1 Giới thiệu

Support Vector Machine (SVM) là một thuật toán học máy được đề xuất bởi Vapnik và cộng sự, thuộc nhóm mô hình học có giám sát (supervised learning). Khác với mô hình CNN – học trực tiếp từ dữ liệu ảnh thô qua nhiều tầng tích chập, SVM hoạt động bằng cách đi tìm một siêu phẳng (hyperplane) tối ưu để phân tách dữ liệu.

SVM là thuật toán đi tìm một siêu phẳng tối ưu để phân tách các lớp dữ liệu thành 2 phía sao cho các biên (margin) là lớn nhất. Các điểm dữ liệu gần nhất mặt phẳng nhất gọi là **Support Vector** – chúng là những điểm tiên quyết để quyết định đâu là mặt phẳng tối ưu nhất cho bài toán.

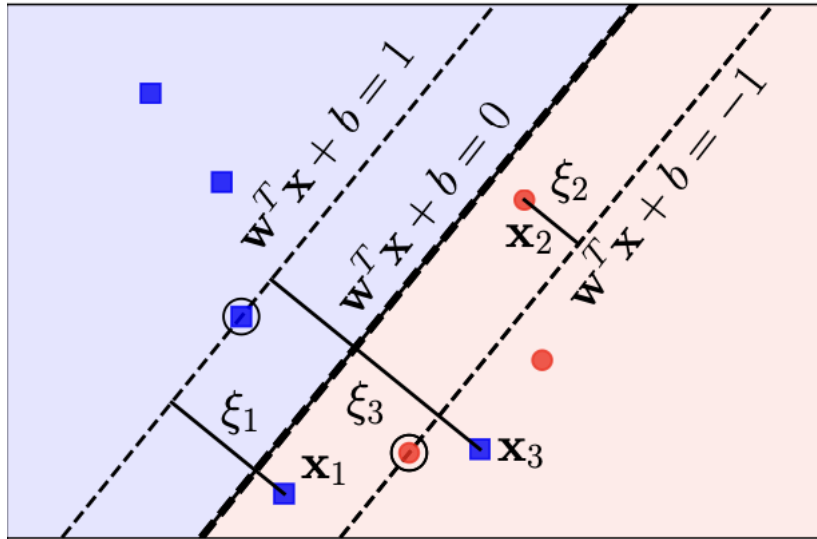
SVM được ứng dụng rộng rãi trong:

- Phân loại nhị phân (Binary Classification)
- Phân loại đa lớp (Multi-class Classification)
- Nhận dạng mẫu (Pattern Recognition)
- Phát hiện bất thường (Anomaly Detection)

3.2.2 Nguyên lý hoạt động

Khác với CNN vốn làm việc dựa trên việc xử lý các lớp xếp chồng lên nhau, SVM hoạt động dựa trên thuyết tối ưu lồi (convex optimization):

Siêu phẳng phân tách (Separating hyperplane)



Hình 11 Minh họa siêu phẳng phân tách và lề mềm trong SVM

Trong không gian đặc trưng $\mathbb{R}^d$, ta quy ước một siêu phẳng có dạng sau:

$$w^T x + b = 0$$

với $w \in \mathbb{R}^d$ là vector pháp tuyến và b là bias. Với nhãn $y \in \{-1, 1\}$, điều kiện để một siêu phẳng phân tách đúng với mọi điểm là:

$$y_i(w^T x_i + b) \geq 1, \forall i.$$

Margin và mục tiêu tối ưu

Margin là khoảng cách giữa hai ranh:

$$\text{margin} = \frac{2}{\|w\|}.$$

Với bài toán SVM thì mục tiêu đặt ra là làm sao để tối đa hoá margin, tương đương tối thiểu hoá $\frac{1}{2} \|w\|^2$ dưới các ràng buộc phân lớp. Ta có bài toán tối ưu hoá (hard-margin):

$$\min_{w,b} \frac{1}{2} \|w\|^2 \text{ subject to } y_i(w^T x_i + b) \geq 1.$$

Đây được gọi là bài toán lồi (convex).

3.2.3 Soft-margin SVM(cho dữ liệu có nhiễu)

Trong thực tế dữ liệu lúc nào cũng bất thường: có nhiễu, outliers hoặc hai lớp không hoàn toàn tuyến tính, nằm tách về hai phía. Lúc này ta cần có Soft-margin SVM cho phép một số điểm vi phạm margin bằng cách thêm biến slack $\xi_i \geq 0$:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0.$$

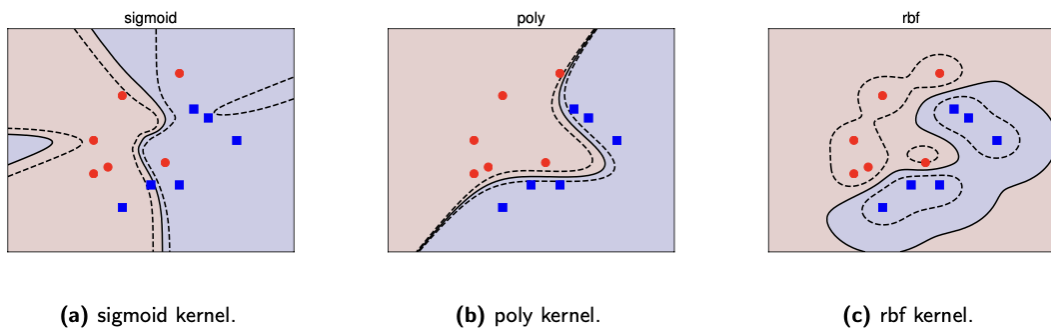
Kết hợp với bài toán lồi ta có:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i,$$

Trong đó $C > 0$ là hệ số phạt cho các vi phạm, khi C lớn, mô hình phạt nặng lỗi hạn chế các lỗi của các điểm dữ liệu; khi C nhỏ mô hình cho phép lỗi nhiều để giữ margin rộng hơn việc này sẽ tốt cho tổng quát. Soft-margin có thể được giải qua bài toán đối ngẫu Lagrange hoặc chuyển về dạng không ràng buộc hinge loss để tối ưu bằng gradient-based methods, thuận tiện cho bài toán lớn.

3.2.4 Kernel SVM (xử lý dữ liệu phi tuyến)

Với các nguồn dữ liệu không đẹp, nhiều dữ liệu không thể phân tách bằng một mặt phẳng trong không gian ban đầu. Ta sử dụng ánh xạ dữ liệu sang một không gian mới có chiều cao hơn, nơi dữ liệu trở nên tuyến tính và phân tách.



Hình 12 Ánh xạ phi tuyến sang không gian chiều cao bằng Kernel Trick

Thay vì tính trực tiếp ánh xạ $\phi(x)$ SVM sử dụng Kernel Function $K(x, z) = \phi(x)^T \phi(z)$ để làm mọi phép toán cần thiết trong không gian mới mà không cần ánh xạ rõ ràng. Một trong số các kernel phổ biến:

- Linear: $K(x, z) = x^T z$
- Polynomial: $K(x, z) = (\gamma x^T z + r)^d$
- RBF (Gaussian): $K(x, z) = \exp(-\gamma \|x - z\|^2)$ — thường được dùng làm mặc định cho bài toán phi tuyến
- Sigmoid: $K(x, z) = \tanh(\gamma x^T z + r)$

Với Kernel thuật toán SVM có thể học các biên phân cách phi tuyến tính phức tạp mà vẫn giữ được lợi thế của SVM là bài toán tối ưu lồi. Tuy, nhiên bài toán cần lưu ma trận kernel (kích thước $N \times N$) vì vậy thuật toán SVM phù hợp với tập dữ liệu vừa và nhỏ.

3.2.5 Bài toán đối ngẫu và support vectors

Sử dụng phương pháp Lagrange, bài toán trên được chuyển sang bài toán đối ngẫu:

$$\max_{\lambda} \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j K(x_i, x_j)$$

Nghiệm λ là sparse: chỉ một số $\lambda_i > 0$. Những điểm có $\lambda_i > 0$ chính là **support vectors** – chính chúng ảnh hưởng đến hàm dự đoán:

$$\sum_{i=1}^N \lambda_i y_i = 0, 0 \leq \lambda_i \leq C.$$

Với bài toán đối ngẫu ta sẽ tối ưu hơn cho module bằng cách lưu các support vector thay vì phải lưu các ma trận lớn, giảm hiệu không gian lưu trữ cho mô hình, tiết kiệm được phần cứng cung cấp cho dự án.

CHƯƠNG 4 : TRIỂN KHAI ỨNG DỤNG

4.1 Giới thiệu tập dữ liệu

Bộ dữ liệu Railway Track Fault Detection trên kaggle là 1 bộ dữ liệu hình ảnh gồm 3 tập Train, Validation và Test, đặc biệt là tập test sẽ được dùng nhiều. Mỗi tập chứa 2 tập Defective (Hư hại) và Non-Defective (Bình thường) của các đường ray ở Bangladesh. Bộ dữ liệu gồm 2.14GB, hơn 360 ảnh chụp. Tuy khá ít nhưng vừa đủ để huấn luyện mô hình.

Dữ liệu có độ đa dạng vừa phải, bao gồm nhiều loại lỗi như:

- Nứt gãy,
- Khuyết thanh ray,
- Biến dạng,
- Sai lệch bề mặt, cùng với nhiều điều kiện môi trường khác nhau.

Một số hình ảnh minh họa của hai lớp được trình bày dưới đây:

Defective



Hình 13 Ảnh minh họa Defective

Non defective



Hình 14 Ảnh minh họa Non Defective

4.2 CNN – Resnet50

Trong phần này, mô hình chính được sử dụng là **ResNet50** – một mạng CNN sâu 50 lớp, được huấn luyện sẵn (pretrained) trên bộ dữ liệu ImageNet. ResNet50 áp dụng kiến trúc **Residual Learning** giúp gradient lan truyền tốt hơn trong mạng sâu, từ đó cải thiện khả năng học và độ chính xác.

Nhằm thích ứng với tập dữ liệu đặc thù của đề tài, mô hình được fine-tune theo **2 giai đoạn (2 phases)**. Mỗi giai đoạn sẽ mở khoá (unfreeze) thêm các tầng sâu hơn trong mạng và huấn luyện lại với learning rate phù hợp.

4.2.1 Xử lý dữ liệu

Dữ liệu hình ảnh ban đầu có kích thước và tỉ lệ khác nhau. Tính toán tập dữ liệu bằng python:

```
sizes = []

for cls in classes:
    folder = os.path.join(train_dir, cls)
    for filename in os.listdir(folder):
        if filename.lower().endswith(('.jpg', '.jpeg', '.png')):
            path = os.path.join(folder, filename)
            with Image.open(path) as img:
                sizes.append(img.size) # (width, height)

# Đếm tần suất từng kích thước
```

```
size_counts = Counter(sizes)
```

Do đó cần được chuẩn hoá trước khi đưa vào mô hình. Các bước xử lý chính bao gồm:

- Resize ảnh về 224×224 pixels
- Chuyển ảnh sang dạng Tensor.
- Chuẩn hoá theo bộ giá trị trung bình và độ lệch chuẩn của ImageNet.
- Tăng cường dữ liệu (Data Augmentation)

Để khắc phục việc thiếu dữ liệu và giúp mô hình tổng quát hóa tốt hơn, nhiều kỹ thuật tăng cường được áp dụng:

- Horizontal Flip
- Random Rotation
- Normalization

Các thao tác này làm tăng sự đa dạng của dữ liệu huấn luyện mà không làm thay đổi nhãn ảnh.

```
#Image Normalize và Augmentation
train_tfms = transforms.Compose([
    transforms.Resize((256, 256)),          # Cố định kích thước
    transforms.RandomCrop(224),             # Crop nhẹ để tạo
    transforms.RandomHorizontalFlip(),       # Lật ngang (an toàn)
    transforms.RandomVerticalFlip(),         # Lật dọc (an toàn)
    transforms.RandomRotation(10),          # Xoay nhẹ
    transforms.ToTensor(),
    transforms.Normalize(mean, std)
])
```

4.2.2 Huấn luyện phase 1

Mục tiêu: Khởi tạo classifier head phù hợp với dataset. Quá trình huấn luyện được triển khai theo mini-batch, sử dụng backpropagation và cập nhật trọng số bằng Adam optimizer.

Chiến lược:

Freeze: Toàn bộ backbone (layer1-4)

Train: Chỉ FC layer (classification head)

Optimizer: Adam với learning rate $1e-4$

Loss: CrossEntropyLoss

Lý do: Giữ nguyên features từ ImageNet, chỉ học phân loại 2 classes mới.

```
# --- PHASE 1: WARM UP HEAD (5 Epochs) ---
for param in model.parameters():
    param.requires_grad = False
for param in model.fc.parameters():
    param.requires_grad = True
```

```
# Loss, optimizer
optimizer = optim.Adam(model.fc.parameters(), lr=1e-4)
criterion = nn.CrossEntropyLoss(label_smoothing=0.01)
```

4.2.3 Huấn luyện phase 2

Mục tiêu: Điều chỉnh các high-level features để phù hợp với *railway track*.

Chiến lược:

Freeze: Không

Train: Toàn bộ ResNet50

Optimizer: Adam với learning rate 1e-4

Epochs: tối đa 20, dừng sớm khi không cải thiện được thêm.

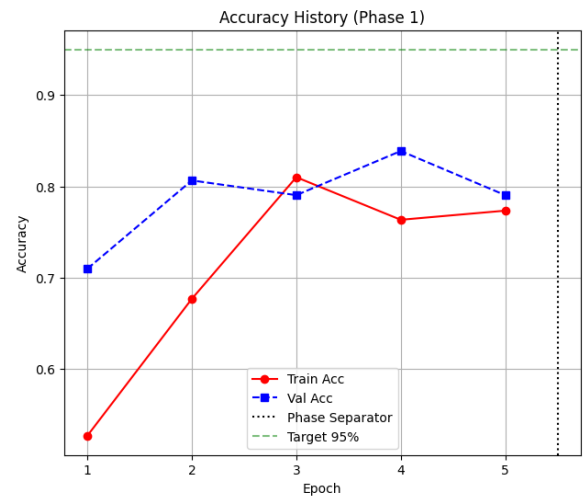
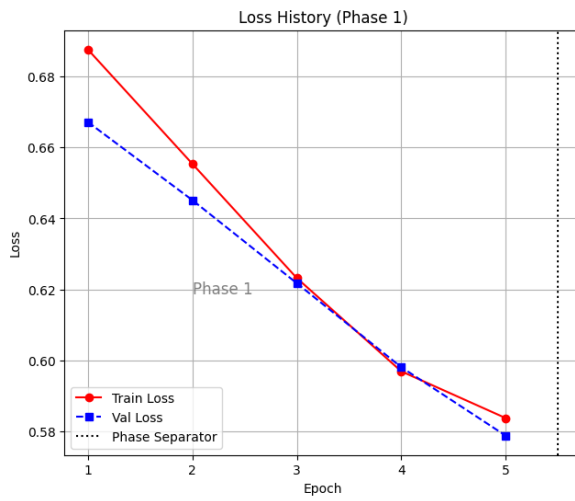
Lý do: điều chỉnh các đặc trưng cấp cao và trung gian cho phù hợp với domain railway track, đồng thời sử dụng early stopping để tránh overfitting.

```
# Mở khóa layer4 + fc để fine-tune (các layer khác giữ nguyên
đóng băng)
for param in model.parameters():
    param.requires_grad = True
```

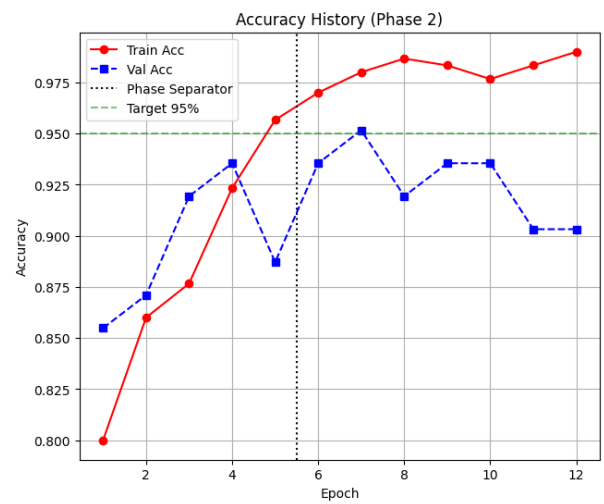
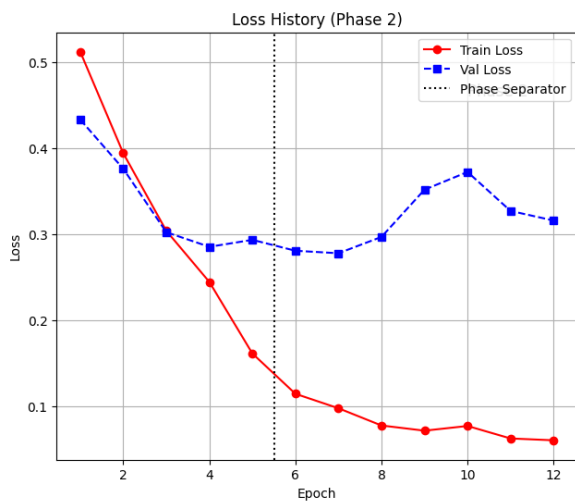
```
# Optimizer LR nhỏ để không phá hỏng backbone
optimizer = optim.Adam(model.parameters(), lr=1e-4,
weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
mode='max', factor=0.1, patience=
best_acc = 0.0
EPOCHS = 20
early_stop = EarlyStopping(patience=5, min_delta=1e-4)
```

4.2.4 Kết quả chạy mô hình

Kết quả chạy ra ở phase 1

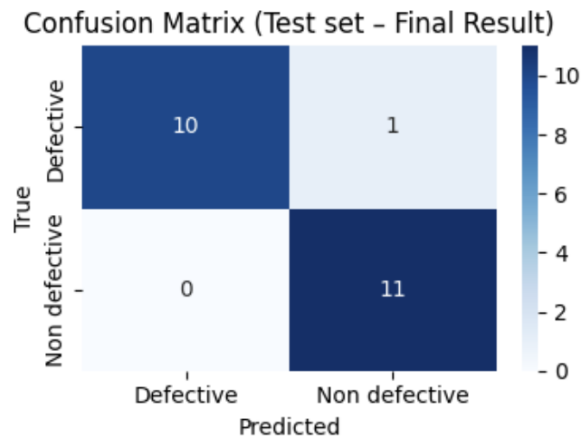


Hình 15 Kết quả huấn luyện phase 1



Hình 16 Kết quả huấn luyện phase 2

	Precision	Recall	F1-Score	Accuracy
Defective	1.000	0.909	0.952	0.955
Non Defective	0.917	1.000	0.957	0.955
Macro avg	0.958	0.955	0.954	0.955
Weighted avg	0.958	0.955	0.954	0.955



Hình 17 Kết quả tốt nhất của tập Test sau fine-tuning

4.3 SVM – Kernel Support Vector Machine

Khác với phương pháp học sâu thuần túy ở phần trước, phần này áp dụng phương pháp hỗn hợp. Chúng tôi sử dụng ResNet50 làm bộ trích xuất đặc trưng để chuyển đổi hình ảnh thành các vector đặc trưng, sau đó sử dụng thuật toán SVM để phân loại.

4.3.1 Trích xuất đặc trưng

Mục tiêu: Chuyển đổi dữ liệu hình ảnh thô thành các vector số học mang ý nghĩa ngữ nghĩa cao, đồng thời giải quyết vấn đề mất cân bằng dữ liệu thông qua tăng cường ngoại tuyến.

Chiến lược:

Backbone: Sử dụng ResNet50 (weights='imagenet') bỏ lớp Fully Connected (include_top=False) và dùng lớp Global Average Pooling để thu về vector đặc trưng kích thước (2048).

Augmentation: Dữ liệu Train được tăng cường trước khi đưa vào trích xuất đặc trưng. Mỗi ảnh gốc được sinh ra thêm 7 biến thể (n_aug=7) bao gồm xoay, dịch chuyển và chỉnh độ sáng.

Lý do: Việc trích xuất đặc trưng giúp SVM làm việc hiệu quả trên không gian dữ liệu chiều cao, giảm tải tính toán so với việc fine-tune toàn bộ mạng CNN, đồng thời tận dụng được khả năng nhận diện đặc trưng mạnh mẽ của ResNet50.

```
resnet_model = ResNet50(
weights='imagenet',
include_top=False,
pooling='avg'
)
```

```

def extract_features(paths, batch_size=16):
    features = []
    for i in range(0, len(paths), batch_size):
        batch_paths = paths[i:i+batch_size]
        imgs = [img_to_array(load_img(p, target_size=(224,
224))) for p in batch_paths]
        arr = preprocess_input(np.array(imgs))
        feats = resnet.predict(arr, verbose=0)
        features.append(feats)
    return np.vstack(features)

# Augmentation
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    brightness_range=[0.8, 1.2],
    fill_mode='nearest'
)

```

```

def extract_augmented_train(paths, labels, n_aug=7):
    features, labs = [], []
    for p, l in zip(paths, labels):
        img = img_to_array(load_img(p, target_size=(224, 224)))
        # Ảnh gốc
        feats_orig = resnet.predict(preprocess_input(np.array([img])), verbose=0)
        features.append(feats_orig)
        labs.append(l)
        # Ảnh augmentation
        for _ in range(n_aug):
            aug = next(datagen.flow(img.reshape(1, *img.shape),
batch_size=1))[0]
            feats_aug = resnet.predict(preprocess_input(np.array([aug])), verbose=0)
            features.append(feats_aug)
            labs.append(l)
    return np.vstack(features), np.array(labs)

# Trích xuất đặc trưng
X_train_feat, y_train_aug =
extract_augmented_train(X_train_paths, y_train, n_aug=7)
X_val_feat = extract_features(X_val_paths) # Validation RAW
X_test_feat = extract_features(X_test_paths) # Test RAW

```

4.3.2 Tối ưu tham số và huấn luyện mô hình

Mục tiêu: Tìm ra siêu phẳng (hyperplane) tối ưu để phân tách hai lớp "Defective" và "Non defective" với sai số thấp nhất.

Chiến lược:

Pipeline: Chuẩn hóa dữ liệu đầu vào (StandardScaler) trước khi đưa vào mô hình SVC để đảm bảo sự hội tụ tốt hơn.

Kỹ thuật K-Fold Cross-Validation: Để đảm bảo tính khách quan và tránh hiện tượng overfitting cục bộ, chúng tôi áp dụng kỹ thuật kiểm định chéo với **K=3 (3-Fold Cross-Validation)**.

- Tập dữ liệu huấn luyện được chia thành 3 phần (folds) bằng nhau.
- Quá trình huấn luyện được lặp lại 3 lần: mỗi lần sử dụng 2 phần để train và 1 phần để validate.
- Kết quả cuối cùng là trung bình cộng của 3 lần chạy, giúp đánh giá chính xác độ ổn định của bộ tham số.

Hyperparameter Tuning: Sử dụng GridSearchCV với kiểm định chéo (Cross-Validation, cv=3) để tìm bộ tham số tốt nhất.

Không gian tham số (Param Grid):

- C: [0.1, 0.5, 1] (Kiểm soát mức độ phạt lỗi - Regularization).
- kernel: ['linear', 'rbf'] (So sánh giữa phân tách tuyến tính và phi tuyến).
- gamma: [0.001, 0.005, 'scale'] (Hệ số nhân cho kernel RBF).

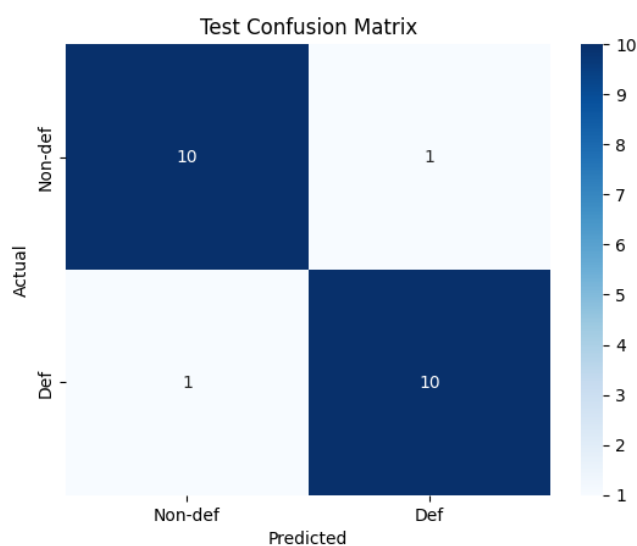
Lý do: GridSearch giúp tự động hóa việc thử nghiệm các tổ hợp tham số, đảm bảo mô hình chọn được cấu hình tối ưu nhất cho tập dữ liệu đường ray.

```
param_grid = {  
    'svc__C': [0.1, 0.5, 1],  
    'svc__kernel': ['linear', 'rbf'],  
    'svc__gamma': [0.001, 0.005, 'scale'],
```

```
pipeline = make_pipeline(StandardScaler(),  
    SVC(probability=True, random_state=42))  
    grid = GridSearchCV(pipeline, param_grid, cv=3,  
    scoring='f1', n_jobs=-1, verbose=2)  
    grid.fit(X_train_feat, y_train_aug)
```


4.3.3 Kết quả chạy mô hình

Class	Precision	Recall	F1-score	Accuracy
Non defective	0.909091	0.909091	0.909091	0.909091
Defective	0.909091	0.909091	0.909091	0.909091
Macro avg	0.909091	0.909091	0.909091	0.909091
Weighted avg	0.909091	0.909091	0.909091	0.909091



Hình 18 TestReport SVM

GridSearchCV đã thử 18 tổ hợp siêu tham số và thực hiện tổng cộng 54 lần huấn luyện (3-fold CV cho mỗi tổ hợp). Bộ tham số tối ưu tìm được là $C=1$, $\text{kernel}=\text{rbf}$ và $\text{gamma}=\text{'scale'}$. Đây là cấu hình giúp mô hình đạt F1-score cao nhất (0.909091) trên 3 lần chia dữ liệu, chứng tỏ mô hình có khả năng tổng quát hóa tốt.

CHƯƠNG 5 : ĐÁNH GIÁ MÔ HÌNH

5.1 Đánh giá mô hình ResNet50

	Precision	Recall	F1-Score	Accuracy
Defective	1.000	0.909	0.952	0.955
Non Defective	0.917	1.000	0.957	0.955
Macro avg	0.958	0.955	0.954	0.955
Weighted avg	0.958	0.955	0.954	0.955

Kết quả trên được ghi nhận từ mô hình ResNet50 tại phase fine-tuning đạt val accuracy cao nhất (0.919), sau đó được đánh giá trên tập test độc lập. Sự chênh lệch này cho thấy mô hình có khả năng tổng quát hóa tốt và không bị overfitting vào tập test.

5.1.1 Accuracy trên tập test

Sau quá trình huấn luyện gồm 2 phase fine-tuning, mô hình ResNet50 đạt độ chính xác tốt nhất là 0.919.

So với các phase trước đó, Phase 1 kết thúc với val accuracy khoảng 0.871.

Điều này chứng minh rằng việc fine-tuning theo từng giai đoạn đã giúp mô hình hội tụ ổn định và tăng dần độ chính xác theo thời gian.

5.1.2 Biểu đồ Loss và Accuracy trong quá trình huấn luyện

Dựa trên log huấn luyện 2 phase, Loss giảm đều và ổn định theo từng epoch. Accuracy cũng tăng dần.

Việc huấn luyện theo từng giai đoạn, bằng cách unfreeze dần từng phần backbone, đã giúp tránh hiện tượng catastrophic forgetting. Đồng thời, phương pháp này tận dụng tốt trọng số pretrained và tối ưu hiệu quả mà không làm mô hình mất ổn định.

5.1.3 Đánh giá quá trình fine-tuning qua từng phase

Trong Phase 1, khi chỉ huấn luyện Fully Connected Layer, độ chính xác trên tập huấn luyện tăng từ khoảng 0.53 lên khoảng 0.81, trong khi độ chính xác tăng từ 0.71 lên khoảng 0.84. Điều này cho thấy mô hình đã học được các đặc trưng cơ bản phù hợp với bài toán từ bộ đặc trưng ImageNet.

Sang Phase 2, khi fine-tune các tầng sâu hơn của backbone, accuracy tiếp tục được cải thiện và đạt giá trị cao nhất là 0.919. Mô hình tại checkpoint này sau đó được đánh giá trên tập Test độc lập và đạt Accuracy 0.955.

Kết luận chung là việc fine-tuning theo hai phase, với chiến lược giữ nguyên backbone ở giai đoạn đầu và chỉ fine-tune các tầng sâu ở giai đoạn sau, giúp mô hình học ổn định và đạt hiệu năng tốt.

5.2 Đánh giá mô hình Kernel SVM

Mô hình SVM được huấn luyện dựa trên đặc trưng 2048 chiều được trích xuất từ ResNet50 (với `include_top = False`, `pooling = 'avg'`) và áp dụng tăng cường dữ liệu (augmentation) với hệ số nhân $7\times$. mô hình SVM cho kết quả tốt trên tập test và đạt độ ổn định trong quá trình tối ưu tham số bằng GridSearchCV.

5.2.1 Đánh giá định lượng

Bảng kết quả dưới đây là Classification Report của mô hình SVM trên tập test.

Class	Precision	Recall	F1-score	Accuracy
Non defective	0.909091	0.909091	0.909091	0.909091
Defective	0.909091	0.909091	0.909091	0.909091
Macro avg	0.909091	0.909091	0.909091	0.909091
Weighted avg	0.909091	0.909091	0.909091	0.909091

Do tập Test của mô hình SVM có kích thước nhỏ (22 ảnh) và phân bố cân bằng giữa hai lớp, số lượng dự đoán đúng và sai của mô hình là tương đương giữa hai lớp. Vì vậy, các chỉ số Precision, Recall, F1-score và Accuracy có giá trị trùng nhau. Đây là hiện tượng bình thường trong các bài toán phân loại nhị phân với tập kiểm tra nhỏ.

5.2.2 Khả năng tổng quát hóa của mô hình

GridSearchCV(3-Fold) giúp tìm ra tham số tối ưu nhất đối với mô hình:

- $C = 1$
- `gamma = 'scale'`
- `kernel = 'rbf'`
- Best CV F1-score = 0.909091

Việc sử dụng GridSearchCV với 3-fold cross-validation giúp đánh giá bộ tham số trên nhiều tập con khác nhau của dữ liệu huấn luyện, từ đó hạn chế overfitting cục bộ và lựa chọn cấu hình có độ ổn định tốt nhất.

5.3 Đánh giá và kết luận chung cho 2 mô hình

Kết quả thực nghiệm cho thấy cả hai mô hình ResNet50 và Kernel SVM đều đạt hiệu năng cao trên tập kiểm tra, với Accuracy và F1-score đều vượt mức 90%. Điều này cho thấy phương pháp học sâu cũng như phương pháp phân loại dựa trên đặc trưng trích xuất đều phù hợp cho bài toán phát hiện khuyết tật đường ray.

Trong đó, mô hình ResNet50 đạt độ chính xác **95.5%**, cao hơn khoảng **4–5%** so với mô hình SVM (**90.9%**). Sự chênh lệch này cho thấy việc fine-tuning trực tiếp mạng học sâu cho phép mô hình thích nghi tốt hơn với đặc trưng dữ liệu cụ thể của bài toán. Do đặc trưng được trích xuất là cố định và không được tinh chỉnh theo nhiệm vụ, mô hình SVM có thể bỏ sót một số đặc trưng tinh tế so với phương pháp fine-tuning trực tiếp mạng CNN.

Tuy nhiên, mô hình SVM vẫn cho kết quả ổn định và cân bằng giữa hai lớp, đồng thời có ưu điểm về độ đơn giản và chi phí huấn luyện thấp hơn. Do đó, tùy theo yêu cầu về độ chính xác và tài nguyên triển khai, cả hai mô hình đều có giá trị ứng dụng thực tế.

CHƯƠNG 6 : KẾT LUẬN

Trong đề tài này, nhóm đã triển khai mô hình Convolutional Neural Network (CNN) với kiến trúc ResNet50 nhằm phân loại tình trạng đường ray dựa trên dữ liệu hình ảnh. Quá trình huấn luyện được thực hiện theo chiến lược fine-tuning 2 giai đoạn, trong đó tập test được sử dụng để theo dõi độ chính xác và lựa chọn phase huấn luyện tối ưu.

Sau khi hoàn tất huấn luyện, mô hình ResNet50 được đánh giá trên tập Test độc lập. Kết quả cho thấy mô hình đạt các chỉ số Accuracy, Precision, Recall và F1-score ở mức cao, chứng tỏ khả năng phân biệt hiệu quả giữa hai lớp “Defective” và “Non-defective”.

Mặc dù đạt kết quả tích cực, đề tài vẫn còn một số hạn chế như kích thước và mức độ đa dạng của tập dữ liệu còn hạn chế, cũng như việc mô hình chưa được kiểm chứng trong môi trường triển khai thực tế hoặc trên dữ liệu video liên tục.

Tổng kết lại, ResNet50 cho thấy là một mô hình phù hợp và khả thi cho bài toán phát hiện lỗi đường ray dựa trên hình ảnh, đồng thời mở ra hướng ứng dụng học sâu trong công tác giám sát và bảo trì hạ tầng đường sắt trong tương lai.

TÀI LIỆU THAM KHẢO

1. Nguyễn, Tuấn Thanh. *Deep Learning cơ bản*. Nguyễn Thanh Tuấn, 2020. *Blog | Deep Learning cơ bản*, <https://nttuan8.com/sach-deep-learning-co-ban/>.
2. Vũ, Tiệp Hữu. *Machine Learning cơ bản*. Vũ Hữu Tiệp, 2018. *Machine Learning cơ bản*, <https://machinelearningcoban.com/ebook/>.
3. Zhang, Aston, et al. *Dive into Deep Learning*. Cambridge University Press, 2023. *Dive into Deep Learning*, <https://D2L.ai>.